Group Assignment IV: Advanced Oracle PL/SQL Programming

Topic: Records, Collections, Cursors, Bulk Processing, Stored Procedures, Functions, and Exception Handling

Course: Database Development with PL/SQL (INSY 8311)

Instructor: Eric Maniraguha | eric.maniraguha@auca.ac.rw **Teaching Assistant:** Afanyu Emmanuel

Assignment Date: Wednesday, October 28, 2026

Class Groups: I, B, C, D

Group Size: Two (2) students per group

Deadline: Your class day next week (2–6 November 2026) at 11:59 PM (STRICT). See the table below.

Submission Deadlines

Class Group	Class Day	Class Time	Room	Submission Deadline
I	Monday	6:00 PM – 8:50 PM	G209	**Monday, 2 November 2026 at 11:59 PM**
B	Tuesday	6:00 PM – 8:50 PM	G207	**Tuesday, 3 November 2026 at 11:59 PM**
C	Wednesday	6:00 PM – 8:50 PM	G207	**Wednesday, 4 November 2026 at 11:59 PM**
D	Friday	1:00 PM – 4:50 PM	G105	**Friday, 6 November 2026 at 11:59 PM**

- Submit by the deadline for **your own class group**. Another class group's later deadline does not apply to you.
- If the two group members attend different class groups, the **earlier** deadline applies.

Important Submission Rules

- **No extensions. Late submission = zero marks.**
- **Do not submit by email.**
- Submit **only** through the Google Form: https://forms.gle/T97Ay3yeKCKBYf7t8
- **One submission per group.** Include the full names and student IDs of **both** group members.
- Your group's GitHub repository **must be public**.
- Make sure your repository contains all required files, screenshots, and documentation.

Instructions to Students

Work in groups of two to design and implement a practical database application using Oracle PL/SQL. The assignment must demonstrate your understanding of records, collections, implicit cursors, explicit cursors, and bulk processing using `BULK COLLECT` and, where appropriate, `FORALL`.

Your implementation must primarily focus on **stored procedures and functions**, with appropriate exception handling throughout the application.

Requirements

1. Records

- Create and use PL/SQL records to store and process related data.
- Demonstrate both user-defined records and `%ROWTYPE` where appropriate.
- Implement record processing within stored procedures or functions.

2. Collections

- Define and use appropriate PL/SQL collection types, such as associative arrays, nested tables, or VARRAYs.
- Demonstrate how collections can store and process multiple values or records.
- Integrate collections into stored procedures and functions.

3. Implicit Cursors

- Use implicit cursors with `INSERT`, `UPDATE`, and `DELETE` statements.
- Demonstrate the use of cursor attributes such as `SQL%ROWCOUNT`, `SQL%FOUND`, and `SQL%NOTFOUND`.
- Implement appropriate exception handling for database operations.

4. Explicit Cursors

- Declare and use explicit cursors to retrieve and process multiple rows.
- Demonstrate cursor processing using `OPEN`, `FETCH`, and `CLOSE`, or a cursor `FOR` loop.
- Implement explicit cursors within stored procedures or functions where appropriate.

5. Bulk Processing

- Use `BULK COLLECT` to retrieve multiple rows into a collection.
- Demonstrate `FORALL` for bulk `INSERT`, `UPDATE`, or `DELETE` operations where appropriate.
- Explain how bulk processing can improve performance compared with row-by-row processing.
- Handle relevant exceptions, including bulk DML errors where applicable.

6. Stored Procedures

- Develop stored procedures that perform meaningful database operations.
- Include input and output parameters where appropriate.
- Integrate records, collections, cursors, or bulk processing into the procedures.
- Include appropriate exception handling and meaningful error messages.

7. Functions

- Develop stored functions that return meaningful results.
- Use parameters, local variables, and appropriate SQL or PL/SQL logic.
- Demonstrate how the functions can be called from an anonymous PL/SQL block or an appropriate SQL statement.
- Handle exceptions appropriately without hiding errors or returning misleading results.

8. Exception Handling

- Include exception handling in all relevant PL/SQL blocks, procedures, and functions.
- Demonstrate predefined exceptions, such as `NO_DATA_FOUND` and `TOO_MANY_ROWS`, where applicable.
- Include user-defined exceptions and `RAISE_APPLICATION_ERROR` where appropriate.
- Ensure that errors are handled meaningfully and that unexpected errors are not silently ignored.

Practical Application

Each group must select a realistic business or institutional scenario, such as:

- Student registration and grading
- Employee management
- Inventory control
- Hospital records
- Banking transactions

The application must include appropriate database tables, relationships, constraints, and sample data. The PL/SQL implementation must demonstrate how the required concepts work together to solve practical problems.

Deliverables

Each group must submit:

1. A brief description of the selected application and its requirements.
2. The database schema, including table definitions, relationships, and constraints.
3. SQL scripts for inserting sample data.
4. PL/SQL source code for records, collections, cursors, bulk processing, stored procedures, and functions.
5. Appropriate exception-handling implementations.
6. Test cases demonstrating successful execution and error handling.
7. A brief explanation of the implementation and performance considerations, especially for bulk processing.
8. A demonstration of the working application during the presentation.

Repository Structure

```
plsql-assignment4-<studentID1>-<studentID2>/
│
├── README.md                        # Group members, application description, how
to run
├── .gitignore
├── 00_schema/
```

```
│   ├── 01_create_tables.sql        # Tables, relationships, constraints
│   └── 02_insert_sample_data.sql
├── 01_procedures/
│   └── <procedure_name>.sql        # One file per stored procedure
├── 02_functions/
│   └── <function_name>.sql         # One file per stored function
├── 03_tests/
│   ├── test_success_cases.sql      # Successful execution
│   └── test_error_cases.sql        # Exception handling
├── screenshots/
│   └── <test_name>_output.png      # Output of each test case
└── docs/
    └── IMPLEMENTATION_NOTES.md     # Implementation and performance explanation
```

How to Run

1. Run `00_schema/01_create_tables.sql`.
2. Run `00_schema/02_insert_sample_data.sql`.
3. Compile the stored procedures in `01_procedures/`.
4. Compile the stored functions in `02_functions/`.
5. Run the test files in `03_tests/`.
6. Verify your results and screenshots.

Group Work and Presentation

- Each group must consist of **exactly two students**.
- Both students must contribute to the design, implementation, testing, and presentation.
- Each student must be able to explain the code and demonstrate an understanding of the implemented concepts.
- Be prepared to answer questions **individually** during the presentation.

Submission

Submission is ONLY through the Google Form.

Submit the link to your group's **public GitHub repository** by **11:59 PM on your class day next week**:

- **Group I (Monday):** Monday, 2 November 2026
- **Group B (Tuesday):** Tuesday, 3 November 2026
- **Group C (Wednesday):** Wednesday, 4 November 2026
- **Group D (Friday):** Friday, 6 November 2026

Submission Form: https://forms.gle/T97Ay3yeKCKBYf7t8

Submission Requirements

- **One submission per group**, with both students' full names and student IDs.
- Repository must be **public**.
- Do **not** submit by email.
- Follow the required repository structure.

- Include all required SQL and PL/SQL files.
- Include screenshots of successful runs and error-handling tests.
- Complete `README.md` and `docs/IMPLEMENTATION_NOTES.md`.
- Both group members must have meaningful commits in the repository.
- Submit the GitHub repository link through the Google Form.

Important Notes

- Use Oracle Database and PL/SQL.
- Organize your code clearly and use meaningful names for tables, variables, procedures, and functions.
- Ensure that all scripts execute correctly and that the application is tested before submission.
- Avoid submitting copied code without understanding it. Each group must be able to explain and justify its implementation.
- Demonstrate both successful operations and appropriate responses to errors.

Final Checklist

- [ ] Group of exactly two students
- [ ] Public GitHub repository with the correct name
- [ ] Application description and requirements in `README.md`
- [ ] Tables, relationships, and constraints created
- [ ] Sample data inserted
- [ ] Records (user-defined and `%ROWTYPE`) used
- [ ] Collections (associative array, nested table, or VARRAY) used
- [ ] Implicit cursors with `SQL%ROWCOUNT`, `SQL%FOUND`, `SQL%NOTFOUND`
- [ ] Explicit cursors (`OPEN`/`FETCH`/`CLOSE` or cursor `FOR` loop)
- [ ] `BULK COLLECT` and `FORALL` used, with bulk DML error handling
- [ ] Stored procedures and functions compile successfully
- [ ] Predefined exceptions, user-defined exceptions, and `RAISE_APPLICATION_ERROR` used
- [ ] Success and error test cases completed, with screenshots
- [ ] Implementation and performance notes completed
- [ ] Both members have meaningful commits
- [ ] GitHub link submitted through the Google Form
- [ ] Both members prepared for the presentation and individual questions

Academic Integrity & AI

This is a **group assignment**. Submit your group's own work. Each member must understand and be able to explain all submitted code.

If you use an AI assistant, mention it in the `README.md` and explain how it was used. Both group members remain responsible for understanding and explaining all submitted code.